Task 5: Same Screen, Different Event-Handling Styles – Currency Converter

Task 5.1 – Four styles

Frame implements ActionListener: register this and implement actionPerformed. Named inner class: create a reusable class implementing ActionListener. Anonymous inner class: pass a new ActionListener object inline. Lambda: pass e -> convert(). All versions should close using WindowAdapter.

Task 5.2 – Changed requirement

Register the same handler on the Convert button and the amount JTextField; pressing Enter then triggers conversion. Use JOptionPane.showMessageDialog for the result. The lambda implementation below uses one shared listener for both triggers.

Task 5.3 – Comparison

Frame listener is simple but couples UI and handler. Named inner class is reusable and clear. Anonymous class is local but verbose. Lambda is concise for a single-method interface. Large applications benefit from named controller classes or Swing Action objects.

Alternative and complexity

A method reference this::convert is concise if signatures match. One shared AbstractAction can be used by a button, menu item and keyboard shortcut. A shared action uses O(1) handler objects; separate listener objects for t triggers may use O(t) space. Adding a trigger is O(1) registration work.

Java Source Code

```
import java.awt.FlowLayout;
import java.awt.event.*;
import javax.swing.*;
public class Converter extends JFrame {
    private final JTextField amount=new JTextField(10);
    private final JComboBox<String> target=new JComboBox<>(new String[]{"USD","EUR"});
    private final JLabel result=new JLabel(" ");
    private static double rate(String c){return c.equals("USD")?0.012:0.011;}
    public Converter(){
        super("Currency Converter");setLayout(new FlowLayout());
        JButton button=new JButton("Convert");
        add(new JLabel("INR"));add(amount);add(target);add(button);add(result);
        ActionListener action=e->convert();
        button.addActionListener(action);amount.addActionListener(action);
        addWindowListener(new WindowAdapter(){public void windowClosing(WindowEvent e){dispose();}});
        setDefaultCloseOperation(DISPOSE_ON_CLOSE);pack();setLocationRelativeTo(null);
    }
    private void convert(){
        try{
            double inr=Double.parseDouble(amount.getText().trim());
            if(inr<0)throw new NumberFormatException();
            String c=(String)target.getSelectedItem();
            String value=String.format("%.2f %s",inr*rate(c),c);
            result.setText(value);JOptionPane.showMessageDialog(this,value);
        }catch(NumberFormatException e){
            result.setText("Invalid amount");
            JOptionPane.showMessageDialog(this,"Invalid amount","Input error",JOptionPane.ERROR_MESSAGE);
```

```
        }
    }
    public static void main(String[] a){SwingUtilities.invokeLater(()->new Converter().setVisible(true));}
}
```

Input Files

valid_input.txt

INR amount: 1000
Target: USD

invalid_input.txt

INR amount: abc
Target: EUR

Output Screenshot

Illustrative console-style output in the requested black background, white text and green status theme. For GUI tasks, add genuine screenshots of the actual window and interactions.



Compile and Run

Open terminal in src, then run:
javac Converter.java
java Converter

README Result Note

Record your Java version, actual execution result, test observations, screenshots and GitHub commit in README.md.